

Sky Analytics Stream

Architettura Distribuita per l'Analisi in Tempo Reale del Traffico Aereo

Flavio Simonelli

Matricola 0365333

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

flavio.simonelli@alumni.uniroma2.eu

Francesco Masci

Matricola 0365258

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

francesco.masci.02@alumni.uniroma2.eu

Sommario—Questo lavoro presenta *Sky Analytics Stream*, una pipeline distribuita ed elastica per l'analisi e il monitoraggio in tempo reale del traffico aereo negli Stati Uniti. L'architettura elabora un flusso continuo di circa 2.2 milioni di eventi storici (da gennaio ad aprile 2025) attraverso una fase di replay accelerata in event-time. Il sistema integra un simulatore ad alte prestazioni sviluppato in Go per l'ingestione dei dati in Apache Kafka, e Apache Flink come motore di stream processing per l'elaborazione distribuita. Tramite un'unica topologia (DAG), Flink esegue concorrentemente tre query analitiche: il calcolo orario delle metriche prestazionali delle compagnie aeree, il tracciamento in tempo reale delle classifiche (Top-10) degli aeroporti con ritardi severi, e la stima della distribuzione dei ritardi tramite l'algoritmo di sketch DDSketch. I risultati analitici e le metriche di telemetria sono storicizzati in InfluxDB tramite Telegraf e visualizzati su dashboard Grafana dedicate. L'architettura è eseguibile localmente con Docker Compose o su un cluster Amazon EC2 con fattori di accelerazione fino a 43.200x (≈ 9.150 eventi/secondo), il checkpointing *exactly-once*, con stato RocksDB incrementale su S3 o HDFS, supporta il recupero dai guasti. La campagna sperimentale confronta parallelismo, disordine temporale e frequenza dei checkpoint.

Index Terms—Apache Flink, Stream Processing, Apache Kafka, Traffico Aereo, Percentili Approssimati, Tolleranza ai Guasti, DDSketch, checkpointing.

I. INTRODUZIONE

L'analisi tempestiva del traffico aereo richiede di aggiornare indicatori e classifiche mentre gli eventi vengono prodotti. In questo contesto non è sufficiente calcolare correttamente un risultato: occorre anche stabilire quando esso possa essere considerato abbastanza completo da essere pubblicato. Ritardi di rete, partizioni inattive e arrivi fuori ordine rendono infatti non coincidenti il tempo associato a un volo e l'istante in cui il sistema lo riceve.

Il progetto affronta questo problema a partire da circa 2,2 milioni di record del Bureau of Transportation Statistics, relativi al periodo 1 gennaio–30 aprile 2025. Il dataset storico viene utilizzato per generare un flusso riproducibile, nel quale la distanza temporale tra gli eventi può essere compressa e il grado di disordine controllato. In questo modo è possibile sottoporre il sistema a condizioni di carico differenti e valutare separatamente gli effetti di velocità di ingresso, parallelismo e ritardo degli eventi.

L'obiettivo è progettare una pipeline distribuita capace di sostenere tre analisi con caratteristiche diverse: aggregazioni orarie sulle compagnie, classifiche degli aeroporti su più

orizzonti temporali e stima della distribuzione dei ritardi. La soluzione deve limitare la quantità di stato mantenuto, gestire esplicitamente i record tardivi, rendere osservabili prestazioni e risultati e recuperare dal fallimento di un nodo senza compromettere la coerenza dell'elaborazione, naturalmente considerando sempre la metrica principale dello stream processing: la latenza.

II. ARCHITETTURA DEL SISTEMA

III. IMPLEMENTAZIONE DELLE COMPONENTI

A. Simulatore di Ingestione in Go

L'ingestione e il replay degli eventi storici in tempo reale sono gestiti da un componente custom sviluppato in linguaggio Go. La scelta tecnologica è motivata dalla necessità di disporre di un agente di ingestione ad altissime prestazioni, caratterizzato da un consumo minimo di risorse e in grado di garantire throughput elevati (fino a oltre 9.000 eventi/secondo) senza introdurre colli di bottiglia o latenze spurie dovute alla Garbage Collection (frequenti in ecosistemi basati su JVM).

Il simulatore implementa diverse funzionalità avanzate e ottimizzazioni ingegneristiche:

- **Ingestione Streaming Zero-Copy ed Integrity Check:** Al primo avvio, l'applicazione verifica l'integrità del dataset scaricato confrontando l'hash SHA-1 calcolato con quello di riferimento. L'archivio `.tar.gz` originale viene letto e decodificato al volo in memoria (*on-the-fly streaming*) tramite la libreria nativa di Go, senza la necessità di estrarre preventivamente su disco circa 2.2 milioni di righe in formato CSV, risparmiando spazio di archiviazione ed eliminando i tempi morti di I/O del disco.
- **Binary Caching in formato GOB:** Al fine di minimizzare i tempi di inizializzazione nelle successive esecuzioni (stress test o tuning della pipeline), gli eventi letti vengono convertiti in strutture tipizzate Go, ordinati cronologicamente rispetto all'event-time calcolato e serializzati in un file di cache binario locale in formato GOB (Go Object Binary). Questo meccanismo riduce il tempo di boot e caricamento del dataset da circa 90 secondi reali a meno di 2 secondi.
- **Gestione del Tempo Fisico ed Event-Time Replay:** Per ciascun volo, l'event-time logico viene ri-

costruito dinamicamente combinando i campi YEAR, MONTH, DAY_OF_MONTH e l'orario programmato CRS_DEP_TIME. Il replay emula la distanza temporale reale tra i voli applicando un fattore di compressione del tempo (*Speedup Factor*) impostabile da configurazione. La temporizzazione è calcolata tramite un meccanismo a precisione millesimale che valuta la differenza relativa (Δt) dell'event-time rispetto all'evento precedente, attendendo il tempo residuo riscaldato prima dell'invio a Kafka.

- **High-Precision Hybrid Waiter (Sleep-Spinlock):** Per riprodurre fedelmente la distanza temporale tra gli eventi, specialmente con alti fattori di accelerazione (dove le finestre temporali logiche di un'ora si contraggono in pochi millisecondi reali), il simulatore non può affidarsi unicamente alle funzioni di *sleep* fornite dal sistema operativo. Queste ultime sono infatti soggette all'imprecisione dello scheduler del kernel e all'overhead dei *context switch*, che introducono latenze variabili nell'ordine dei 10-15 ms. Per ovviare a questo limite, è stata implementata una strategia di attesa ibrida. Se la durata dell'attesa supera una determinata soglia critica (*spin threshold*, impostata a 15 ms), il thread esegue una sleep cooperativa rilasciando la CPU; non appena il tempo residuo scende sotto la soglia, il thread effettua un **spinlock** (attesa attiva ad alta frequenza). Questo impedisce al sistema operativo di sospendere il processo, assicurando una precisione di rilascio nell'ordine dei microsecondi e garantendo l'uniformità del throughput.
- **Generazione di Disordine (Out-of-Order Injection):** Al fine di stressare e validare le strategie di Watermarking e la gestione dei dati tardivi (*allowed lateness*) di Apache Flink, il simulatore implementa un decoratore basato su un algoritmo probabilistico. Con una probabilità definita $P \in [0, 1]$, un record viene selezionato e il suo istante di pubblicazione viene dislocato in avanti nel tempo di un offset casuale limitato da un valore massimo configurabile (δ_{max} in minuti logici). La sequenza di eventi risultante viene infine ri-ordinata sul tempo di pubblicazione reale prima dell'invio, riproducendo accuratamente i tipici disallineamenti di rete di uno stream distribuito.

IV. INFRASTRUTTURA DI DEPLOYMENT

V. VALUTAZIONE SPERIMENTALE

VI. CONCLUSIONI E SVILUPPI FUTURI

RIFERIMENTI BIBLIOGRAFICI

- [1] L. Fernando, H. Bindra, and K. Daudjee, "An experimental analysis of quantile sketches over data streams," in *Proceedings of the 26th International Conference on Extending Database Technology (EDBT '23)*, pp. 1–12, 2023.
- [2] C. Masson, J. E. Rim, and H. K. Lee, "DDSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees," *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 2195–2205, 2019.

APPENDICE A
UTILIZZO DI STRUMENTI DI INTELLIGENZA ARTIFICIALE GENERATIVA

APPENDICE B
RAPPRESENTAZIONI GRAFICHE E ALLEGATI